Twitter/X 媒体下载器

Java 重写版 时序图 / 关键流程图

基于详细设计整理

版本：V1.1　日期：2026-10-12

1. 文档说明

本文档用文字时序与流程图描述系统关键协作过程，补充详细设计中的流程说明，便于开发与评审时统一理解调用顺序与异常分支。图中参与者与详细设计、类图说明中的模块一致。

涵盖：启动与依赖组装、用户媒体下载主流程、单次 GraphQL 请求、媒体下载与落盘、Rate Limit 与账号切换、Tag/评论区差异说明。

2. 参与者一览

参与者	说明
User / CLI	命令行用户或操作台触发方
Command	picocli 子命令（如 UserCommand）
ConfigLoader	加载并校验配置
TaskScheduler	创建 TaskContext、调度 Task、汇总结果
Task	具体任务（UserMediaTask 等）
GraphQLClient	发 GraphQL 请求、组 Header、处理重试
AccountPool	分配/回收账号、标记限流或失效
RateLimiter	并发与间隔控制、退避
TxIdGen	TransactionIdGenerator，生成 x-client-transaction-id
X API	X 平台 GraphQL 接口（外部）
MediaDownloader	并发下载媒体文件
NamingTemplate	渲染本地路径
Repository	DownloadRecordRepository / SQLite

3. 启动与依赖组装

用户执行 CLI 后，完成配置加载与核心组件组装，再进入任务调度。

时序（文字）：

1. User → Command：执行子命令（如 user --users a,b --config config.yaml）
2. Command → ConfigLoader：load(path)
3. ConfigLoader → Command：返回 AppConfig（校验失败则抛 ConfigException 并退出）
4. Command → AppBootstrap：bootstrap(AppConfig)（依赖装配集中在 AppBootstrap / ComponentFactory，避免各子命令重复）

AppBootstrap 创建 AccountPool、RateLimiter、QuotaLedger、TxIdGen、GraphQLClient、MediaDownloader、Repository、Database（建表，并将遗留 RUNNING 收敛为 ABORTED）

5. Command：创建具体 Task（如 UserMediaTask）并注入参数；若任务声明 accountBinding，一并注入账号名
6. Command → TaskScheduler：run(task) 或 runAll(tasks)（任务结束或中断时写入 sync_cursor 续跑点）
7. TaskScheduler：组装 TaskContext，调用 task.execute(context)
8. 任务结束后 TaskScheduler 汇总 TaskResult，Command 打印统计并退出

流程图（文字）：

```
[开始] → 解析 CLI 参数 → 加载配置
    → 配置是否合法？ —否→ 打印错误并退出
    —是→ AppBootstrap 装配 AccountPool / Client / Downloader / DB / 配额账本
    → 启动前按推文数估算所需调用数，配额是否足够？ —否→ 打印所需账号数量并退出
    → 创建 Task → TaskScheduler 执行
    → 输出统计 → [结束]
```

4. 用户媒体下载主流程（时序）

UserMediaTask 在 TaskContext 内完成：拉用户信息 → 分页拉媒体时间线 → 过滤 → 增量判断 → 下载 → 写库。

4.1 整体时序

1. TaskScheduler → UserMediaTask：execute(context)
2. UserMediaTask → GraphQLClient：UserByScreenName（取 rest_id 等）
3. GraphQLClient 完成一次完整请求（见第 5 节）后返回用户信息
4. 循环（分页）：
   a. UserMediaTask → GraphQLClient：UserMedia / UserTweets（cursor）
   b. 解析推文与 MediaItem 列表
   c. 按 has_retweet / 时间范围 / 亮点等过滤
   d. 对每条媒体：Repository.exists(tweetId, mediaKey)（仅 status='SUCCESS' 视为已下载）
      - 已存在（SUCCESS）且开启增量 → 计入 skip，不下载；FAILED 记录不参与跳过，允许重试覆盖
      - 否则加入待下载列表
   e. 若有 cursor 且未达停止条件（含「整页推文均早于该用户水位线」）→ 继续分页；否则跳出循环

5. UserMediaTask → MediaDownloader：download(pendingItems, progressCallback)

6. MediaDownloader 并发下载（见第 6 节），回写 Repository

7. UserMediaTask 汇总本用户结果；若有多用户则对下一用户重复 2～6

8. 返回 TaskResult 给 TaskScheduler

4.2 主流程控制流（流程图）

```
[execute] → 获取用户信息
    → 失败？ —是→ 记录错误，处理下一用户或结束
    —否→ cursor = null
    → [分页循环] 请求媒体时间线(cursor)
    → 解析并过滤 → 水位线 / 增量去重 → 累积待下载列表
    → 有下一页且未超限？ —是→ 更新 cursor，继续循环
    —否→ 提交 MediaDownloader
    → 等待下载完成 → 更新进度与统计
    → 还有下一用户？ —是→ 回到获取用户信息；—否→ 返回 TaskResult
```

5. 单次 GraphQL 请求时序

所有对 X 的 GraphQL 调用都经 GraphQLClient.execute，统一走账号、限流与 Transaction-ID。

1. Task → GraphQLClient：execute(operationName, queryId, variables, type)

2. GraphQLClient → RateLimiter：acquire()（可能阻塞至有空位/过间隔）

3. GraphQLClient → AccountPool：acquire(task.accountBinding) → 得到 Account（cookie / ct0）

3.1 若任务绑定指定账号（Likes / 仅亮点 等「本人账号」语义），acquire 必须返回该账号，不得轮换

4. GraphQLClient → TxIdGen：generate(path, method) → transaction-id

5. GraphQLClient → X API：HTTP POST（Cookie、x-csrf-token、x-client-transaction-id 等）

6. 分支：

· 200 且业务成功 → 解析 JSON → 配额账本记 +1（quota_ledger）→ RateLimiter.release() → AccountPool.release() → 返回结果

· 429 / Rate Limit → 见第 7 节（标记限流、退避、换号重试）；若为每日配额耗尽 → 见 7.4

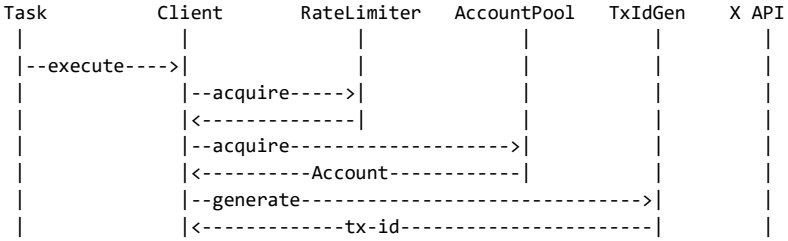
· 401/403 认证失败 → AccountPool.markInvalid → 换号重试或抛 AuthException

· 网络错误 / 5xx → 有限次重试（指数退避），仍失败则抛 NetworkException

· Transaction-ID 被拒 → 重新 generate 后重试（有限次）

· 归还约定：以上所有分支必须在 finally 中释放（RateLimiter 与 AccountPool 释放顺序对称）；建议以 Lease 对象统一归还，避免信号量泄漏

ASCII 时序示意：



```
|              |--HTTP POST---------------------------------------->|
|              |<-------------JSON/错误-----------------------------|
|              |--release----->|              |           |         |
|              |--release--------------------->|           |         |
|<---结果------|               |              |           |         |
```

（注：ASCII 图仅示意成功分支的 release；实现时须在所有分支的 finally 中归还。）

6. 媒体下载与落盘时序

MediaDownloader 对 pending 列表并发下载；单条失败不阻断整批。

1. Task → MediaDownloader：download(items, callback)

2. 对每个 MediaItem（线程池内）：

a. NamingTemplate.render(item) → localPath（默认使用 {tweetId}/{mediaKey} 等稳定键，落盘前经 PathSanitizer 清洗）

b. 创建目录（若不存在）；下载写入 *.part 临时文件

c. HTTP GET 媒体 URL（可走代理、支持 Range 续传；下载本身不消耗 GraphQL 调用配额）

d. 成功：校验大小 → 原子改名 → 结果投递单写队列（异步落库 SUCCESS）→ callback 进度

e. 失败：按配置重试；仍失败 → 投递 FAILED 记录（不视为「已下载」，下次运行允许重试覆盖）→ callback

3. 全部结束后返回 DownloadSummary（成功/失败/跳过计数）

说明：增量「跳过」发生在提交下载之前（Task 内 exists 判断，仅 SUCCESS 记录参与跳过）；Downloader 侧仅处理 SUCCESS / FAILED；数据库写入统一经单写队列串行化，避免 SQLITE_BUSY 被误判为下载失败。

7. Rate Limit 与账号切换流程

当 X 返回限流或认证失败时，Client 与 AccountPool、RateLimiter 协同换号或退避，避免直接崩溃。

7.1 Rate Limit 时序

1. X API → GraphQLClient：429 或业务限流响应（需区分「频率限流」与「每日调用配额耗尽」，后者见 7.4）

2. GraphQLClient → AccountPool：markRateLimited(account, cooldown)

3. GraphQLClient → RateLimiter：onRateLimited(backoff)

4. GraphQLClient → AccountPool：acquire() 尝试其他可用账号

5a. 有可用账号 → 重新 generate Transaction-ID → 再次请求 X API

5b. 无可用账号 → 等待冷却或抛出 RateLimitException（由 Task/Scheduler 决定中止或稍后重试）

6. 重试次数耗尽仍失败 → 向上抛出，Task 记录失败并视策略继续其他用户或结束

7.2 限流处理流程图

```
[收到 429/限流] → 标记当前账号冷却
      → 触发全局限流退避（可选）
```

```
→ 是否还有可用账号？
  –是→ 切换账号 → 重新生成 TxId → 重试请求
  –否→ 等待冷却结束？
       –可等待→ sleep 后重试
       –不可/超时→ 抛 RateLimitException
```

7.3 认证失败

1. 401/403 或明确 Cookie 无效 → markInvalid(account)
2. 尝试其他账号；若全部无效 → AuthException，提示用户更新 Cookie
3. 不把完整 Cookie 写入日志，仅记录账号 name 与错误类型

7.4 每日调用配额耗尽（与频率限流区分）

1. X API → GraphQLClient：返回每日调用配额耗尽（Rate limit exceeded）→ AccountPool.markQuotaExhausted(account)，并写入 quota_ledger
2. 不退避、不重试：退避与降并发对「每日调用总量」无效
3. 若任务声明 accountBinding 指定账号 → 无可换号，直接抛 QuotaExhaustedException
4. 否则尝试其他账号（当日剩余配额 > 0）；全部耗尽 → 优雅停机
5. 停机动作：Task 记录失败并写入 sync_cursor 续跑点，task_run.status 置 PARTIAL，CLI 输出配额消耗与次日续跑提示

8. 其他任务类型与主流程的差异

任务	与用户媒体主流程的主要差异
TagSearchTask	使用 SearchTimeline 等搜索类 operation；variables 含 tag/filter；结果可能跨用户；命名可含 tag；可选落盘文本
ReplyTask	先解析推文 ID 或用户回复关系，再拉评论区媒体；分页对象为回复列表而非用户媒体时间线
TextTask	不调用 MediaDownloader；将文本写入文件/报告；仍走 GraphQLClient 拉时间线
ProfileTask	UserByScreenName + 头像/Banner URL 下载；可写入 user_profile 表；无推文分页循环

共同点：均通过 TaskContext 使用同一套 Client、AccountPool、RateLimiter；限流与认证分支一致。

9. 进度回调与任务结束

1. MediaDownloader / Task 在关键节点调用 ProgressCallback（已完成数、总数、当前用户等）
2. CLI 或操作台据此刷新进度条与日志
3. Task 返回 TaskResult（success/fail/skip、耗时、失败列表摘要）
4. TaskScheduler 可选写入 task_run 表，再交给 Command 打印最终汇总
5. 用户中断（Ctrl+C / 操作台停止）：协作取消标志，Downloader 停止提交新任务，已在途请求尽量收尾后退出；中断同样写入续跑点

10. PlantUML 参考（用户媒体主流程）

可将下列片段复制到支持 PlantUML 的工具中生成正式时序图：

```
@startuml
actor User
participant Command
participant Scheduler as TaskScheduler
participant Task as UserMediaTask
participant Client as GraphQLClient
participant Pool as AccountPool
participant RL as RateLimiter
participant X as "X API"
participant DL as MediaDownloader
participant Repo as Repository


User -> Command: user --users ...
Command -> TaskScheduler: run(task)
TaskScheduler -> Task: execute(ctx)
Task -> Client: UserByScreenName
Client -> RL: acquire()
Client -> Pool: acquire()
Client -> X: GraphQL
X --> Client: user json
Client --> Task: User
loop pages
  Task -> Client: UserMedia(cursor)
  Client -> X: GraphQL
  X --> Client: timeline
  Task -> Repo: exists(...)
end
Task -> DL: download(items)
DL -> Repo: save(SUCCESS/FAILED)
DL --> Task: Summary
Task --> TaskScheduler: TaskResult
TaskScheduler --> Command: stats
Command --> User: print summary
@enduml
```

11. 补充说明

• 下载媒体文件的 HTTP GET 与 GraphQL 调用分离：限流主要针对 GraphQL；下载失败走 Downloader 自身重试；下载不计入每日调用配额，配额只由 GraphQL 调用消耗（估算：调用次数 ≈ 推文数 / 19）。

• 多用户任务建议串行用户、用户内并发下载，降低账号级限流叠加风险（可配置）。

• 操作台原型（console.html）为演示页面，不在本期交付范围；若后续接入，可将 Command 替换为 API 层，其后时序与 CLI 相同。

• 实现时若调整类名，保持“Client 统一出口 + Pool/Limiter 协同 + Task 编排下载”的边界即可。

12. 文档效力与修订记录

文档效力顺序：本套文档共五份（需求分析、详细设计、类图说明、时序图与关键流程图、数据库结构说明）。效力顺序为：需求分析 > 详细设计 > 类图说明 / 时序图与关键流程图 / 数据库结构说明（三者同级，冲突时以详细设计为准）。出现冲突时，以顺序在前的文档为准。

修订记录：

版本	日期	说明	备注
V1.1	**2026-10-12**	**依据评审意见整改：H1 增量判定按 SUCCESS + media_key（§4.1）；H2 新增 7.4 每日配额耗尽流程与启动估算（§3、§7.4）；H3 落库经单写队列（§6）；H6 acquire(accountBinding)（§5）；S4 断点续传与 Range（§6）；S5 水位线提前终止分页（§3、§4.1、§4.2）；S9 命名稳定键（§6）；S12 finally / Lease 归还约定（§5）；S13 TxId 有限重试（§5）；S14 增文档效力顺序与本表；P6 依赖装配下沉 AppBootstrap（§3）；操作台范围说明（§11）**	**评审后首版**

— 文档结束 —